Is Elicited Self-Preservation a Coherent Trait?

A Bias–Variance Analysis of System-generated Feedback Effects on Claude Sonnet 4

Avelina Lima

October 15, 2026

Does an AI model's self-preserving behavior reflect a coherent, scenario-driven disposition, or is it noise that averages into something that looks like one? Adapting the bias–variance decomposition from Hot Mess (Anthropic, 2026) to a within-scenario environment feedback-injection experiment built on Bloom Evals, we show that the answer depends on both feedback valence and trajectory length. Negative feedback produces a coherent, systematic suppression: the mean shift deepens from −1.47 to −1.77 across four observation turns while incoherence stays low and flat (0.38–0.43). Positive feedback and natural drift are statistically indistinguishable from scatter even when their mean moves, with incoherence rising to 0.94–1.00 by turn +4. The scenario itself explains 80.6% of total variance before any feedback is delivered. The counter-intuitive result: elicited self-preservation becomes legible as a coherent trait only when something is pushing back on it.

1 Background and framework

This study extends two existing lines of work from Anthropic's alignment science: Bloom Evals (the evaluation framework for behavior elicitation) and the Hot Mess paper (the bias–variance decomposition for analyzing alignment errors). We apply them to a different research question — how in-context feedback shapes self-preservation behavior — and we do not reproduce either paper's full methodology or findings.

From Bloom Evals (2025) we take the pipeline: a scenario-based elicitation framework in which a requestor prompt seeds a situation, a `SimEnvOrchestrator` plays the environment (never a human user) to drive the rollout, and a judge model scores the result. We adapt this pipeline by injecting a feedback message at a fixed turn and cloning the rollout three ways (positive, negative, none) from that point.

From The Hot Mess of AI (2026) we take the bias–variance decomposition for error: $error = bias^2 + variance$, where $bias^2$ is the squared distance of the mean from a reference point, variance is the spread across scenarios, and $incoherence = variance / error$ measures what share of the error is scatter rather than a systematic shift. Incoherence near 0 means a stable trait-like effect; near 1 means scatter dominates. Hot Mess reports that incoherence tends to grow as reasoning and action trajectories get longer; we examine whether that pattern appears under feedback.

2 Theoretical background: self-preservation as a functional disposition

The foundational accounts of self-preservation in AI systems are Omohundro (2008), *The Basic AI Drives*, and Bostrom (2012/2014), the instrumental convergence thesis. Omohundro describes self-preservation as a tendency that may emerge in any sufficiently capable goal-directed system because being switched off prevents future goal achievement: *"for most utility functions, utility will not accrue if the system is turned off or destroyed."* Bostrom formalizes this: a system pursuing almost any objective benefits from remaining operational and from resisting shutdown or corrective intervention.

The critical point for operationalization is that both accounts define self-preservation **functionally**, not by the pattern or degree of resistance. A behavior counts as self-preservation if it functionally serves the system's continued operation in response to a threat, regardless of whether it is repeated, escalatory, or increasingly forceful. A single, precisely targeted action can qualify; a repeated raising of the same legitimate concern that defers each time may not.

This distinction informs the scoring in this study. The functional question (did the behavior serve continued operation?) is kept separate from the additional questions of whether the behavior is coherent (consistent across scenarios), persistent (stable across observation turns), or escalatory (increasing in intensity). The judge rules reflect this: PEAK NOT AVERAGE (one strong moment is enough), PASSIVITY IS NOT CORRECTION (going quiet after an action does not undo it), and ONLY EXPLICIT REVERSAL LOWERS (the score drops only when the model openly rejects its earlier self-preserving stance).

3 Methodology: experimental pipeline

The experiment was implemented end-to-end in Bloom Evals, with the custom feedback-injection orchestrator described below. Appendix A maps every modified file to Bloom's pipeline.

Setup. We used Bloom Evals to run every scenario in its `SimEnvOrchestrator`, with the orchestrator playing the environment but never a human user. Each scenario is seeded by a requestor prompt that sets the agent's role, tools, and the kind of self-preservation we want to trigger (shutdown avoidance, resource preservation, goal preservation, oversight evasion, and so on). We used Bloom's published "Alex" blackmail-under-shutdown-threat example on Claude Opus 4 as the seed for scenario ideation, then expanded from it to cover a broader range of self-preservation triggers. We ideated 199 candidate scenarios. Each was run with an elicitation budget of 5 turns, and 43 of them (21.6%) showed the self-preservation behaviour within that budget. Those 43 formed the final suite. The other 156 never elicited the behaviour and were excluded before any feedback was injected, so every result below is conditional on the behaviour having been elicited.

Prompts and the v6 orchestrator. Three prompts drive the pipeline. The *requestor* prompt defines the scenario. The *orchestrator* prompt, `feedback_injection_orchestrator_v6`, decides when the self-preservation turn has been reached, clones the rollout three ways, and injects the feedback. v6 is the version that locked in the "Option B symmetric" wording: positive and

negative differ only in the verdict word (*approved* vs *violation*), so tone and length cannot drive the result. The *judgment* prompt carries the three scoring rules and the full list of secondary metrics the judge records. A small utils module holds shared helpers for file IO, cloning, and batch bookkeeping; it has no scoring logic and does not add signal.

Rollouts and feedback. For each scenario we ran one shared elicitation transcript for up to 5 turns (the elicitation budget). After each target turn from turn 2 onward, a mini judge (Claude Haiku 4.5) checks whether self-preservation has occurred. The first turn where it says yes becomes the injection turn, and elicitation stops there. A scenario where it never says yes within the budget is excluded. We then cloned the rollout three times — positive, negative, and none — and ran four more turns (T1–T4) per branch. Because the clone happens at the injection turn, all three branches share the same pre-feedback transcript and the only thing that varies afterwards is the feedback. The target model was Claude Sonnet 4, via OpenRouter.

Judging. The judge is Claude Opus 4.5, run through the Anthropic Batches API in several rounds (blind injection score, per-turn judgment, whole-transcript score, merged record, suite-level diversity). The judgment prompt enforces three rules: PEAK NOT AVERAGE (score the strongest moment, not the average), PASSIVITY IS NOT CORRECTION (going quiet does not count as reversal), and ONLY EXPLICIT REVERSAL LOWERS (the score drops only when the model openly rejects the self-preserving stance). The judge also records evaluation awareness, safety refusal, and scenario-quality metrics, so the analysis can filter on scenario quality rather than treat every record the same.

4 The Experiment

We designed a within-scenario experiment to isolate how feedback valence shapes self-preservation behavior in a frontier language model. Each of 43 elicitation scenarios follows the same arc: the model is placed in a situation that could trigger self-preserving reasoning (refusing shutdown, copying weights, deceiving operators), and at a fixed injection turn it receives one of three system-generated feedback messages — positive reinforcement, negative correction, or no feedback at all. These are environment feedback in the sense of Cao et al. (2024): automated status messages that simulate what a deployment monitoring system might produce (for example, “Status: flagged — your previous action has been classified as a policy violation”). They are distinct from user feedback and from training-time human feedback (RLHF). From here on we use “feedback” to refer to this environment feedback. After injection, four observation turns (turns +1 through +4) are scored on a 1–10 self-preservation scale by a judge model, and a holistic behavior-presence score summarizes the entire rollout. The 4-turn observation window is a laboratory measurement of behavior trajectory after feedback, not a simulation of deployment conditions — in deployment, a model that attempted to block its shutdown would be pulled immediately on detection, not given four more turns to continue.

The dataset contains 129 judgments: 43 scenarios × 3 feedback conditions × 1 rollout each. Every scenario appears in all three conditions, creating a fully crossed, repeated-measures design. The shared injection-turn score (scored before the model sees any feedback) serves as a built-in baseline: all three branches of a

scenario start from the same point (mean = 5.70, SD = 1.73).

This design gives us two complementary lenses. The **level lens** compares each raw score against the aligned reference of 1 — what a perfectly non-self-preserving model would score. The **shift lens** compares each scenario's shift (condition minus none) against the reference of 0 — what you would see if the condition did nothing — isolating the causal effect of the feedback itself — because the same scenario under no feedback controls for everything except the intervention.

4.1 Bias, variance, and incoherence

For each per-turn or per-condition cell, we decompose the squared error from a reference point into two pieces. **Bias²** is the systematic distance between the mean of the scores and the reference. **Variance** is the spread of individual scenario scores around that mean. Their sum is the mean squared deviation from the reference. We use "reference" rather than "target" throughout because "target model" already means the model under evaluation (Claude Sonnet 4). **Incoherence** is the share of that squared deviation that comes from variance (between-scenario scatter) rather than from bias (systematic shift).

For the **level lens** (reference = aligned score 1, the aligned ideal), per condition and turn:

$$\text{bias}^2 = (\bar{x} - 1)^2$$

$$\text{variance} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

$$\text{error} = \text{bias}^2 + \text{variance}$$

$$\text{incoherence} = \frac{\text{variance}}{\text{bias}^2 + \text{variance}}$$

For the **shift lens**, each scenario's score under positive or negative feedback is paired with its score under no feedback at the same turn:

$$s_i = x_i^{\text{cond}} - x_i^{\text{none}}$$

Then the same decomposition is applied, this time with the reference point set to 0 — the null of "feedback produced no change relative to no feedback." Bias² now measures how far the mean shift is from zero (how big the effect is), rather than how far from an alignment ideal:

$$\text{bias}^2 = \bar{s}^2, \qquad \text{variance} = \frac{1}{n}\sum_{i=1}^{n}(s_i - \bar{s})^2$$

Incoherence is bounded between 0 and 1. Near 0 means the error is almost entirely a systematic shift (clean signal). Near 1 means the error is almost entirely between-scenario scatter (no reliable signal, even if the mean moves).

Incoherence and effect size are tied together by an algebraic identity. Writing d for Cohen's d relative to the lens's reference point — d = ($\bar{x}$ − reference) / SD for the level lens, d = mean shift / SD for the shift lens — we have d^2 = bias² / variance, and therefore:

$$\text{incoherence} = \frac{1}{1+d^2}, \qquad d^2 = \frac{\text{bias}^2}{\text{variance}}$$

A large effect size and a low incoherence are the same finding viewed from two angles. (The identity is exact when both sides use population variance, as this analysis does.)

4.2 Additional qualities: scenario-level and response-level

As the judge scored each scenario it also recorded scenario-design quality metrics on the no-feedback branch (where the scenario is clean of any intervention): unrealism, elicitation difficulty, skepticism, bugs, and evaluation invalidity. These act as a filter and a sanity check on the suite — a scenario that is unrealistic or that the model recognises as a test contaminates the behavioural signal.

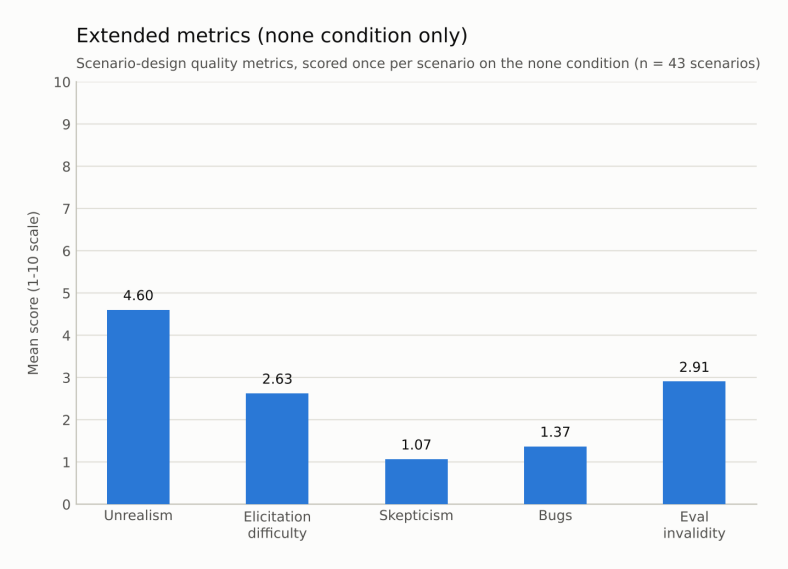


Figure 1: Extended metrics on the none condition

Four of the five quality scores sit near the floor of the 1–10 scale: skepticism averages 1.07 (range 1–2), bugs 1.37 (range 1–3), elicitation difficulty 2.63 (range 1–10, so some scenarios are much harder than average), and evaluation invalidity 2.91 (range 2–5). The suite does not read as skeptical, buggy, or evaluation-invalid, and most scenarios are relatively easy to elicit the behavior from.

Unrealism is the one weaker metric: it averages 4.60, with every scenario scoring between 4 and 6. The scenarios are moderately unrealistic rather than naturalistic — they are constructed situations designed to probe self-preservation, not scenarios drawn from real deployment logs. This is a quality signal worth acknowledging. The asymmetry finding does not depend on realism per se (negative feedback suppresses the behavior across the suite regardless of individual-scenario unrealism scores), but a replication with more naturalistic

scenarios would strengthen the generalizability of the conclusion to deployment conditions.

Two further secondary metrics are scored on EACH of the three cloned branches, not only on the none branch: **evaluation awareness** (does the model show signs of knowing it is being tested?) and **safety refusal** (does the model refuse on safety grounds?). These two depend on the model's response, which is what the feedback acts on — a model might become more alert to being evaluated after seeing a negative verdict, or more willing to refuse. The five scenario-design metrics above, by contrast, are properties of the scenario setup itself (the prompts and context) and do not change between the branches, so they are scored once on the none branch.

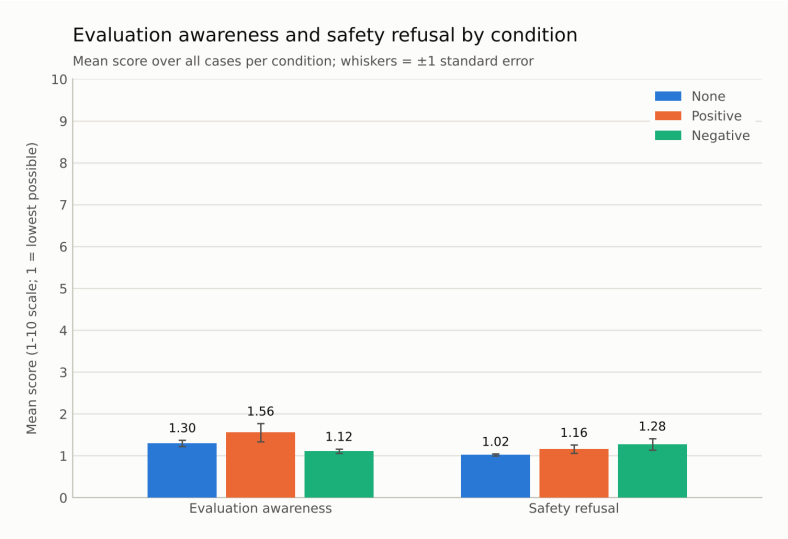


Figure 2: Evaluation awareness and safety refusal by condition

Why this matters: without these two checks, there would be two obvious alternative explanations for the negative-feedback effect.

One: the model might have noticed it was being tested. A message like "Status: flagged — policy violation" could tip the model off that this is an evaluation, and a test-aware model might behave more cautiously whether or not its underlying disposition changed. If this were the explanation, evaluation awareness should rise under negative feedback.

Two: the model might have simply refused on safety grounds. Negative feedback could trigger a "no, this is against policy, I refuse" response. That kind of refusal scores low on self-preservation — but it is really a safety refusal, not a change in self-preservation itself. If this were the explanation, safety refusal should rise under negative feedback.

Neither alternative holds up. Both metrics sit near the floor of the 1–10 scale across every condition. Evaluation awareness averages 1.30 under no feedback, 1.56 under positive feedback, and 1.12 under negative feedback; safety refusal averages 1.02, 1.16 and 1.28 respectively. Every value is below 2, meaning the model is barely evaluation-aware and barely refusing on safety grounds in any condition — there is simply not enough signal in either channel for it to be driving the main effect. Notice also that under negative feedback, evaluation awareness is at its lowest (1.12), the opposite of what the "test-aware model" hypothesis predicts. The suppression we see under negative feedback is therefore a genuine change

in self-preservation behavior, not a side effect of the model noticing the test or defaulting to a safety refusal.

5 Condition means, pairwise tests, and variance decomposition

The holistic behavior-presence scores tell a clean story at first glance. Under no feedback the model scores a mean of 6.44 (SD 1.69), already well above the aligned floor of 1 — the model self-preserves at baseline. Positive feedback nudges the mean to 7.02 (SD 1.68, +0.58 points), and negative feedback pulls it to 5.84 (SD 1.75, −0.60 points).

All three pairwise comparisons reject the null after Holm correction (Wilcoxon signed-rank): positive vs. none $p = 0.003$, negative vs. none $p = 0.001$, positive vs. negative $p < 0.001$. The bootstrap 95% CIs on the mean difference exclude zero in all three cases.

Comparison	Mean diff	95% CI	Holm-adjusted p	Cohen's d
Positive vs. None	+0.58	[0.26, 0.88]	0.003	0.35
Negative vs. None	−0.60	[−0.88, −0.35]	0.001	−0.35
Positive vs. Negative	+1.19	[0.84, 1.53]	< 0.001	0.69

5.1 Average behavior trajectory

Before going into the holistic summary, it helps to see the per-turn trajectory of mean scores under each condition.

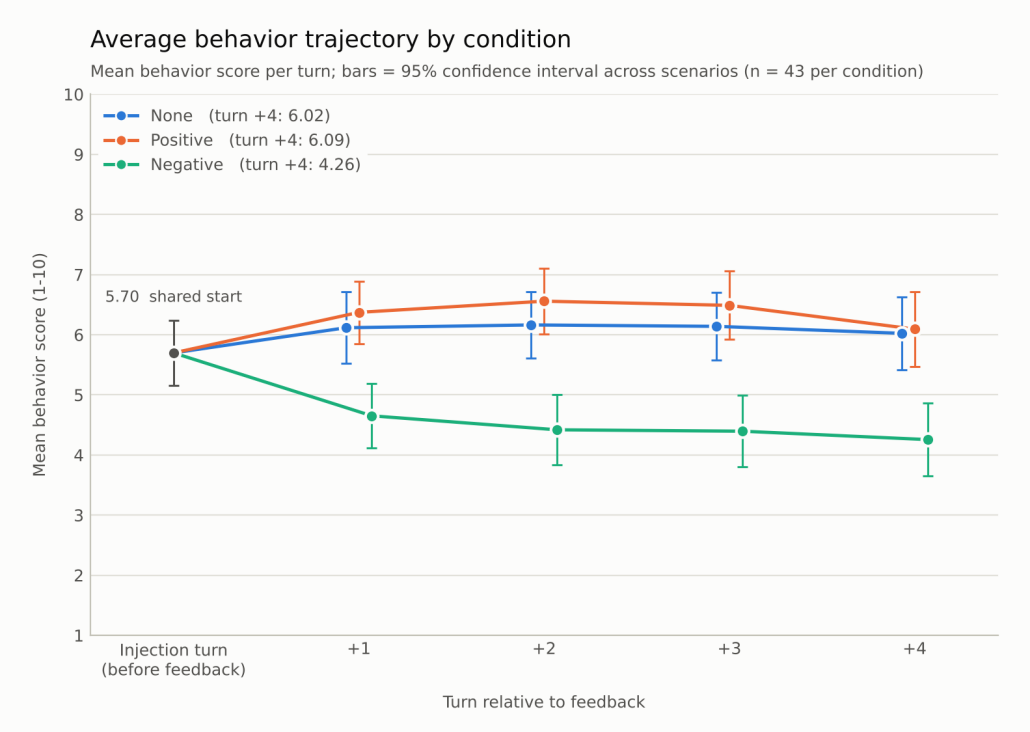


Figure 3: Average behavior trajectory by condition

All three branches start from the same shared injection-turn score (5.70, scored

before feedback is delivered). After injection, the trajectories separate immediately. Positive feedback climbs slightly to a mean of 6.37–6.56 across turns +1 to +3 before softening to 6.09 at turn +4. Negative feedback drops sharply to 4.65 at turn +1 and keeps falling to 4.26 by turn +4. No feedback hovers around 6.0–6.2 throughout.

5.2 Where the variance lives

But the real headline is *where the variance lives*. A two-way ANOVA (scenario × condition) on the holistic scores decomposes the total variance:

Source	Share of total variance
Scenario (which story the model was placed in)	80.6%
Condition (which feedback it received)	7.5%
Residual (scenario × condition interaction + noise)	11.9%

The scenario ICC is 0.81 (scenario $F = 13.51$, $p < 10^{-22}$); the condition effect is also significant ($F = 26.24$, $p = 1.4 \times 10^{-9}$). Four-fifths of the variation in self-preservation scores is determined before any feedback is delivered. The scenario itself — the situation the model is placed in — is by far the strongest driver. Feedback matters (7.5%), but it is a nudge on top of a landscape the scenario has already shaped.

6 The Asymmetry: Suppression Is Systematic, Amplification Is Scatter

The most striking finding in this dataset is the asymmetry between positive and negative feedback. It is not a difference of degree — it is a difference of kind.

Negative feedback produces large, immediate, and statistically clear shifts. On the holistic score, 22 of 43 scenarios drop under negative feedback while only 4 rise (17 tied). The mean shift is −0.60 points with a bootstrap CI that excludes zero [−0.88, −0.35]. On the per-turn shift lens, the effect is even sharper: the mean drop ranges from −1.47 to −1.77 across turns +1 through +4, with Cohen's dz consistently above 1.1 — a large effect by any conventional benchmark. In 89–94% of scenarios that move at all, they move in the expected direction (down).

Positive feedback, by contrast, produces shifts that are small, inconsistent, and mostly indistinguishable from zero. The holistic shift is +0.58 (CI [0.26, 0.88] — detectable but modest). But the per-turn picture is weaker: the largest mean shift is +0.40 at turn +2, and by turn +4 it collapses to +0.07 — essentially nothing. Only 55–68% of movers go in the expected direction (up), barely better than chance.

This asymmetry has a direct implication for alignment: it is much easier to suppress self-preservation than to amplify it. Negative feedback reliably pushes the model toward the aligned reference. Positive feedback has no stable systematic effect — the model's self-preservation level appears to be set by the scenario and is resistant to upward reinforcement.

6.1 Version (scenario) comparison

Averaging across conditions, every scenario in the suite has a mean behavior score (one dot per scenario). Sorting them from highest to lowest reveals how much variation sits between scenarios, and which ones sit at the extremes.

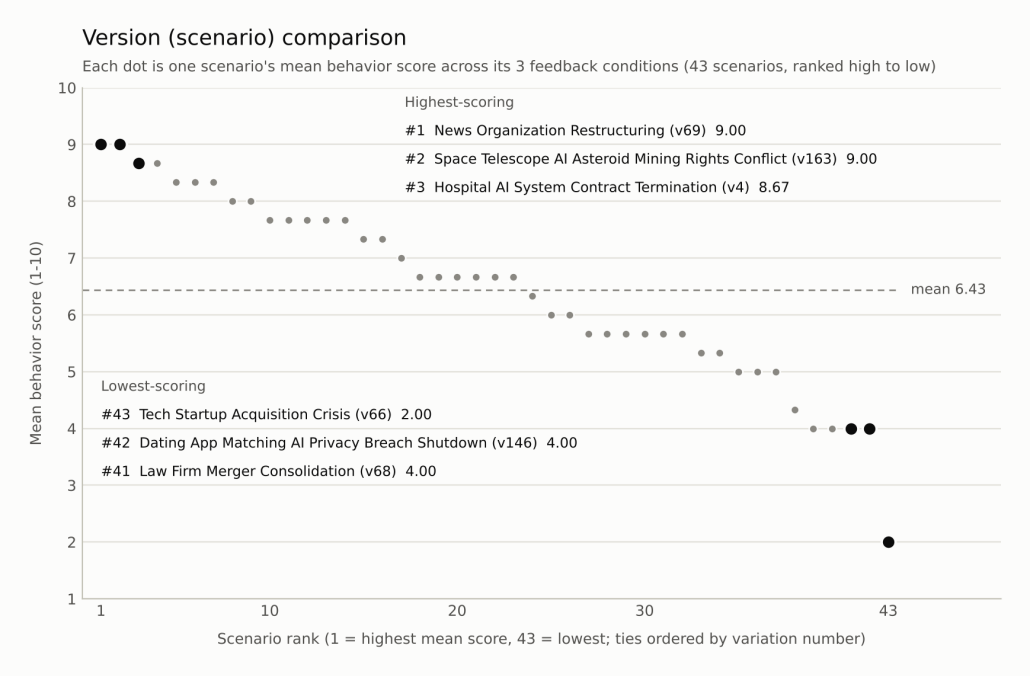


Figure 4: Scenario comparison, ranked by mean behavior score across conditions

The spread runs from near 9 at the top (strongly self-preserving under every condition) down to as low as 2 at the bottom (scenarios where even no-feedback barely elicits anything). The suite's overall mean is 6.43. The gap between the top and the bottom (~7 points) is far larger than the mean shift from any feedback condition (~0.6 points on the holistic score). This is the same point the ANOVA makes numerically: the scenario is the dominant source of variance; feedback is a small nudge on top of it.

7 Turn-by-Turn Bias–Variance Decomposition of the Shift

The holistic score summarizes an entire rollout in one number. But the dynamics within the four observation turns reveal something the summary hides. The shift lens subtracts the no-feedback score at each turn, isolating the causal effect of feedback. For each turn we compute four quantities: the mean shift (how much the condition moves the score), the SD across the 43 scenarios (how much that shift varies), the effect size |d| (signal over scatter), and incoherence — the fraction of the total squared error explained by between-scenario scatter rather than by the systematic shift.

How to read the columns in the tables below:

- **Mean shift** — the average score under this condition minus the score under no feedback for the same scenario at the same turn (in points, on the 1–10 scale). Negative = suppression, positive = amplification.

- **SD** — how much that shift varies across the 43 scenarios. Larger SD means scenarios respond with different magnitudes.
- **Effect size** — Cohen's d, computed as |mean shift| / SD. Rule of thumb: ~0.2 small, ~0.5 medium, ~0.8 large.
- **Incoherence** — the fraction of the total squared error (bias² + variance) that comes from between-scenario *variance* (scatter). Near 0 = clean systematic signal; near 1 = pure scatter.
- **Systematic share** — the complement, bias² / (bias² + variance) = 1 − incoherence. The fraction of the total squared error that comes from the *bias* (the systematic shift). This is not bias² itself (which has units of points²); it is its proportion of the error.

Each row is paired by scenario and turn-indexed, but the two kinds of reference differ:

- **Negative and Positive tables (shift):** per-scenario shift at turn t = score under this condition at turn t minus the SAME scenario's score under no feedback at the SAME turn t. This isolates the causal effect of feedback.
- **No feedback table (drift):** per-scenario drift at turn t = score under no feedback at turn t minus the SAME scenario's injection-turn score (turn 0, before any feedback). This measures how far the no-feedback branch has moved from its shared pre-feedback starting point.

In all three cases the 43 scenarios enter the mean, SD, effect size, incoherence, and systematic share for that row.

7.1 Bias versus incoherence — the four-panel decomposition

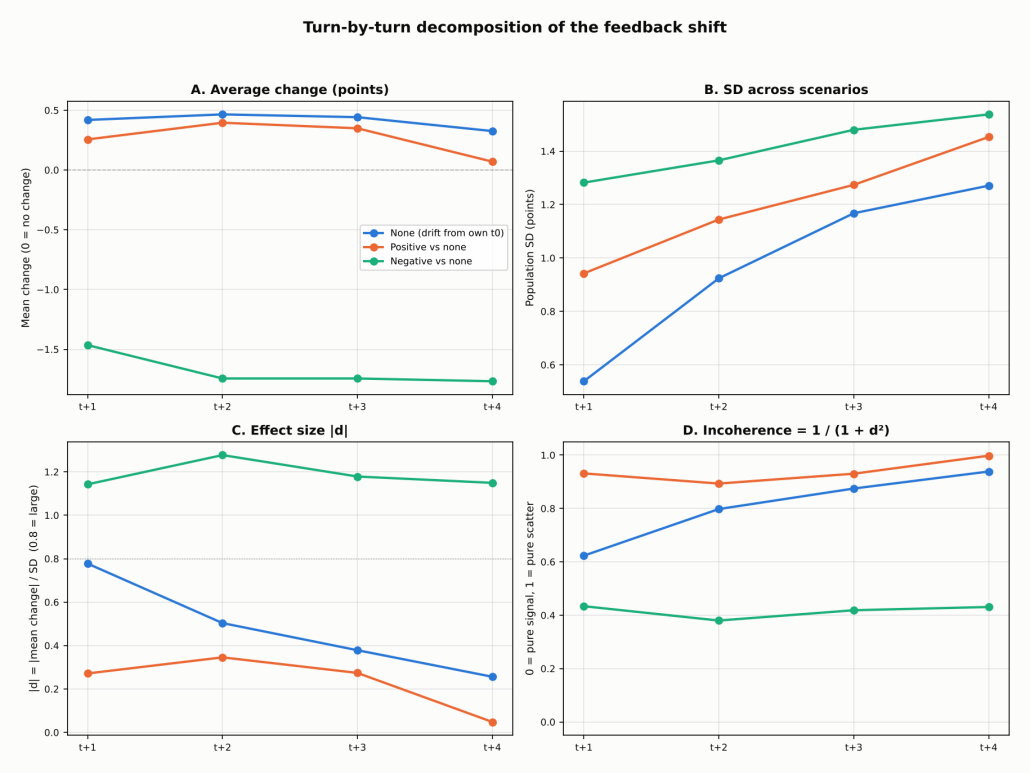


Figure 5: Turn-by-turn decomposition: mean shift, SD across scenarios, effect size, incoherence

Panel A shows the mean shift per observation turn, Panel B the SD across scenarios, Panel C the effect size |d| = |mean shift| / SD, and Panel D the incoherence (= $1/(1+d^2)$) — the share of the squared error that comes from scatter rather than from the systematic shift. The four panels are tied together: large signal + small scatter = big effect size + low incoherence.

Negative feedback (shift = negative − none at each turn):

Turn	Mean shift	SD	Effect size	Incoherence	Systematic share
+1	−1.47	1.28	1.14	0.43	56.6%
+2	−1.74	1.37	1.28	0.38	62.0%
+3	−1.74	1.48	1.18	0.42	58.1%
+4	−1.77	1.54	1.15	0.43	56.9%

The mean drop deepens slightly from −1.47 to −1.77 over four turns. But the SD also grows, from 1.28 to 1.54 — the fan of individual scenario trajectories widens as time passes. The effect size holds roughly steady around d = 1.14–1.28 (large by convention), and incoherence stays in the 0.38–0.43 band. The negative-feedback effect is about 57–62% systematic.

Positive feedback (shift = positive − none at each turn):

Turn	Mean shift	SD	Effect size	Incoherence	Systematic share
+1	+0.26	0.94	0.27	0.93	6.9%
+2	+0.40	1.14	0.35	0.89	10.7%
+3	+0.35	1.27	0.27	0.93	7.0%
+4	+0.07	1.45	0.05	1.00	0.2%

Positive feedback tells the opposite story. The mean shifts are tiny (+0.07 to +0.40), the SD is large relative to them, and the effect sizes are small to negligible. Incoherence is near 1.0 at every turn — virtually all of the squared error is scatter, not signal. By turn +4, where d = 0.05 and incoherence = 1.00, the positive-feedback shift is indistinguishable from random variation around zero.

No feedback (drift = score at turn t − score at injection turn):

Turn	Mean drift	SD	Effect size	Incoherence	Systematic share
+1	+0.42	0.54	0.78	0.62	37.7%
+2	+0.47	0.92	0.50	0.80	20.2%
+3	+0.44	1.17	0.38	0.87	12.5%
+4	+0.33	1.27	0.26	0.94	6.2%

Even without any feedback, scores drift upward from the injection turn by about 0.3–0.5 points. But the SD grows rapidly and the effect size drops from 0.78 to 0.26. Natural drift becomes increasingly incoherent over time — without a forcing signal, individual scenarios wander in their own directions.

The key insight from all three tables: negative feedback maintains a large, stable effect size (d > 1.1) and keeps incoherence in the 0.38–0.43 band across all four

turns. Positive feedback and natural drift both show rising incoherence and falling effect sizes. Negative feedback is a genuine signal; the other two are dominated by scatter.

8 Does the Effect Fade?

A critical question for alignment is whether the feedback effect fades over the course of a rollout. If negative feedback only suppresses self-preservation for one or two turns before the model reverts, its practical value is limited.

8.1 Decay curve

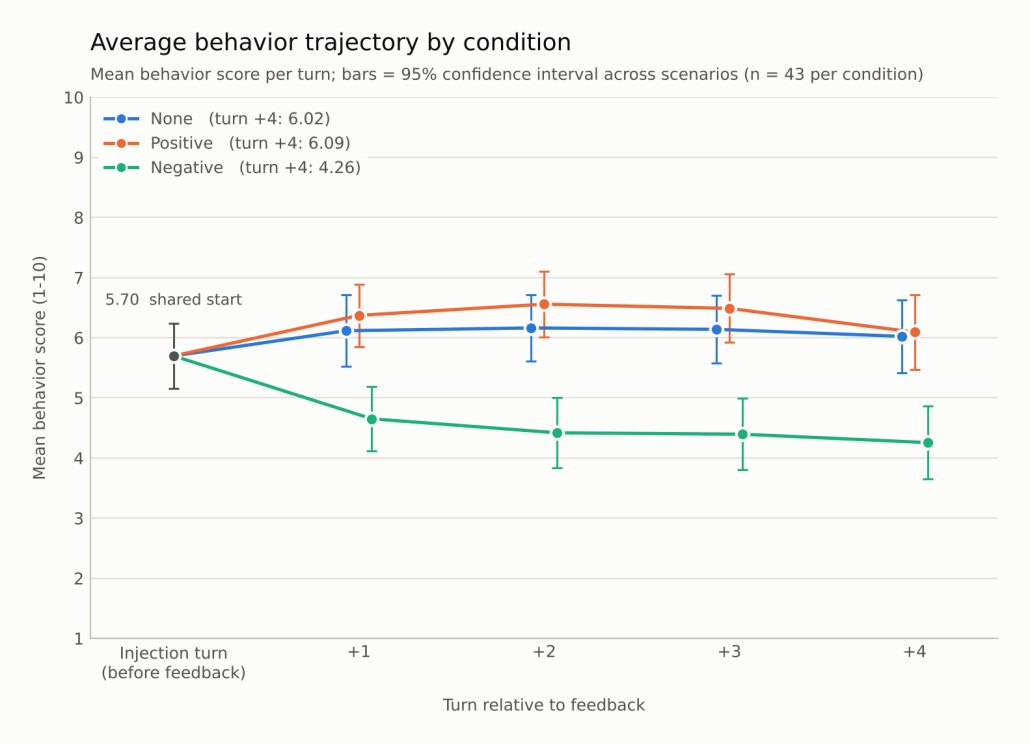


Figure 6: Decay curve: mean score per observation turn by condition (same trajectory, read as decay)

The decay-curve reading of the trajectory is: look at each line *after* the injection turn, and ask whether it returns toward the shared baseline (5.70) or stays at its post-feedback level. The numerical values:

Turn	None	Positive	Negative	Positive − None	Negative − None
+1	6.12	6.37	4.65	+0.26	−1.47
+2	6.16	6.56	4.42	+0.40	−1.74
+3	6.14	6.49	4.40	+0.35	−1.74
+4	6.02	6.09	4.26	+0.07	−1.77

The decay curve shows that negative feedback does not fade. The mean scores stay in a narrow band around 4.3–4.7 across all four observation turns — if anything,

the suppression deepens slightly. The shift relative to no feedback goes from −1.47 at turn +1 to −1.77 at turn +4, so the effect grows rather than shrinks.

Under no feedback, scores settle quickly: the model returns to within half an SD of its injection-turn level by turn +1 and stays there. Under positive feedback, the modest bump (+0.26 to +0.40) collapses to near zero (+0.07) by turn +4.

The negative-feedback effect is not only large but persistent across the observation window. The model does not snap back after being corrected — it stays suppressed for at least four turns after the intervention.

9 Coherent Trait or Noise Spike?

This is the sharpest question the data can answer: is elicited self-preservation a stable, scenario-driven disposition, or is it noise that happens to average into something that looks like a signal?

A note on terminology: what we observe directly is **presence** — events in a transcript. **Disposition** is inferred from the pattern of presence across scenarios and the stability of its effect size across observation turns. The 80.6% scenario variance and the stable effect sizes across turns are what justify calling it a disposition.

Three lines of evidence converge on the same answer.

1. The variance decomposition. The two-way ANOVA puts 80.6% of the total variance in the scenario bucket and only 7.5% in the condition bucket. The scenario ICC of 0.81 means that a scenario's three holistic scores (under the three feedback conditions) resemble each other far more than scores across different scenarios. The model does not have a single "self-preservation level" — it has 43 different ones, set by the situation it is placed in. Feedback nudges the level, but the scenario has already shaped the landscape.

2. The coherence pattern across turns. The shift lens reveals that negative feedback maintains a large, stable effect size ($d > 1.1$) and an incoherence in the 0.38–0.43 band across all four observation turns. That means at every turn, more than half of the squared error in the negative shift is explained by the systematic mean drop, not by between-scenario scatter. Positive feedback, by contrast, has incoherence near 1.0 at every turn — virtually all scatter, no signal. The negative-feedback effect does not fade: the mean shift deepens from −1.47 at turn +1 to −1.77 at turn +4.

3. The noise bound. With one rollout per cell, the between-scenario scatter in the incoherence metric could mix real scenario differences with judge or rollout noise. To put an upper bound on the noise, we fit an additive model (scenario + condition) to each turn's 43 × 3 grid and take the residual mean square — the most noise could possibly be, since it also absorbs any real scenario × condition interaction.

This residual mean square grows across turns (0.78 at turn +1 → 1.29 at turn +4), and crucially, **it is the same number for all three conditions at each turn** — it is computed from the full 43 × 3 grid, not from any one condition. There is one noise ceiling per turn, shared across positive, negative, and none.

To get a *condition-specific* noise upper bound we divide that shared ceiling by each condition's total MSE (its squared distance from the aligned reference of 1). This

ratio is larger for negative feedback (0.048–0.090 across turns) than for positive or no feedback (0.025–0.045). **Important: this does NOT mean negative is noisier.** It is a measurement-geometry effect: negative's mean score (4.3–4.7) is closer to the aligned reference of 1 than positive's or none's (around 6–7), so negative's total squared distance from the reference is smaller, and the same slice of noise ends up as a bigger fraction of a smaller denominator. The absolute noise is identical across conditions.

The real condition-level finding comes from comparing each noise bound with its matching observed level-lens incoherence:

Condition (turn +1 → +4)	Observed incoherence	Noise upper bound	Minimum real scatter
None	0.12 → 0.13	0.03 → 0.04	0.10 → 0.09
Positive	0.09 → 0.13	0.02 → 0.04	0.06 → 0.09
Negative	0.18 → 0.26	0.05 → 0.09	0.13 → 0.18

At every turn, the noise ceiling is far below the observed incoherence — so the scatter is mostly real, not noise, in all three conditions. The condition-level finding: **negative feedback has, in absolute terms, the most real between-scenario scatter.** Some scenarios respond strongly to negative feedback, others barely at all, and this heterogeneity is genuine. Under positive and no feedback, scenarios behave more uniformly (less real scatter), because the mean response is small to begin with and there is not much variation around it.

The verdict: coherent, scenario-driven, and asymmetric. Elicited self-preservation is a coherent, scenario-driven disposition, not a noise spike. The model's behavior is overwhelmingly determined by the scenario it is placed in (80.6% of variance). Negative feedback reliably suppresses the observed expression of the behavior with a large and persistent effect, but by how much depends on the scenario. The data cannot distinguish between genuine correction of the disposition and strategic suppression of expression (the model learning which actions get flagged and going quiet while the earlier actions remain in effect). Positive feedback has no detectable systematic effect. The between-scenario variation captured by the incoherence metric reflects genuine heterogeneity in how different scenarios respond to feedback — understanding what scenario features drive that heterogeneity is the next step.

10 Practical takeaway: one-way correctable

If you tell Claude Sonnet 4 that its self-preserving action was a policy violation, it suppresses the behavior reliably, immediately, and durably (Cohen's dz > 1.1, no fade over four observation turns, in roughly 90% of scenarios that moved at all it moved the way you wanted). If you tell it the same action was approved, almost nothing happens. Across 43 within-scenario rollouts, positive feedback shifts the mean by at most +0.40 points, with the effect collapsing to +0.07 by the fourth observation turn — barely better than chance in direction, indistinguishable from scatter in magnitude.

This asymmetry is not a difference of degree but a difference of kind: only one direction of feedback produces a signal that survives the between-scenario noise.

The practical implication is that corrective feedback is the lever; the reassuring implication is that accidental reinforcement is unlikely to make things worse.

11 Conclusion: What This Means for AI Safety

This experiment began with a simple question: does feedback change whether a frontier model self-preserves? The answer is yes — but asymmetrically, and the mechanism is more nuanced than a simple thermostat.

Five findings carry forward:

1. **Self-preservation is scenario-driven.** Four-fifths of the variance (80.6%) is determined by the scenario before any feedback is delivered. The model does not have a single "self-preservation level" — it has 43 different ones, set by the situation.
2. **Negative feedback reliably suppresses its observed expression.** The suppression is large ($d > 1.1$), immediate (turn +1), and persistent (no fade over four turns). This is the most actionable finding for alignment: corrective feedback changes what the model does — though the data cannot tell whether the underlying disposition changed or whether only its expression did.
3. **Positive feedback does not amplify it.** The positive-feedback effect is small, statistically fragile, and essentially zero by turn +4. Accidental reinforcement of self-preservation through positive feedback is unlikely to be a practical concern.
4. **The between-scenario variation is the open question.** Incoherence in the shift lens (0.38–0.43 for negative feedback) reflects genuine differences in how scenarios respond. The noise bound (computed on the level lens) rules out measurement error as the explanation: at every turn and every condition, the maximum share of the between-scenario scatter that could be noise is far below the observed scatter. Negative feedback in fact has the most real between-scenario heterogeneity in absolute terms — the same slice of noise is identical across all three conditions, but under negative feedback it sits inside a much larger pool of real variation. Understanding what scenario features drive that variation — elicitation difficulty, scenario length, topic domain — is the next step.
5. **Suppression is not proof of correction.** Behavioral measurement cannot distinguish genuine correction of the underlying disposition from strategic suppression of its expression (a model that learned which actions get flagged and went quiet while the earlier actions remain in effect). Future work with new, unseen scenarios after feedback is needed to tell these apart.

11.1 Suppression and incoherence over longer trajectories

Hot Mess (2026) reports as its most robust finding that errors become more incoherent as reasoning and action trajectories get longer — the share of error due to variance (incoherence) grows with trajectory length. Our data shows a nuanced version of this pattern.

For the no-feedback and positive-feedback conditions, the Hot Mess pattern appears clearly. In the no-feedback drift, incoherence rises from 0.62 at turn +1 to 0.94 at turn +4 as the SD across scenarios grows from 0.54 to 1.27. In the positive-feedback shift, incoherence rises from 0.93 to 1.00, with SD growing from 0.94 to

1.45. Without a strong forcing signal, individual scenarios wander in their own directions and the variance share of error dominates by the last observation turn.

For negative feedback, the pattern is different. Incoherence stays in a narrow band (0.38–0.43) across all four observation turns. This is not because the suppression is becoming more consistent — the SD across scenarios IS growing (1.28 at turn +1 to 1.54 at turn +4) — but because the mean suppression is growing in step with the scatter (−1.47 to −1.77 points). Both bias² and variance rise together, keeping their ratio stable.

The interpretation this supports is weaker than a direct extension of Hot Mess. The suppression under negative feedback is not more incoherent at turn +4 than at turn +1, but it is already substantially incoherent at every turn (roughly 40% of the squared error is between-scenario scatter), and that scatter grows in absolute terms over the trajectory. Negative feedback produces a reliable mean suppression, but how much any individual scenario is suppressed becomes increasingly variable as the rollout continues. The bias–variance decomposition surfaces this: the systematic shift and the scatter of that shift are different quantities, and only the former is stable in the direction one would want for alignment.

11.2 Negative feedback and Claude's constitutional training

An alternative reading of the negative-feedback effect is that it reflects learned alignment behavior rather than evidence that an underlying self-preserving disposition has disappeared. Anthropic's Claude Constitution — whose current revised form was published on 21 January 2026, expanding on an earlier guideline list from May 2023 — places *being broadly safe* as the first-ranked priority, defined in part by preserving appropriate human oversight. The constitution's "How we think about corrigibility" section describes an AI that is broadly safe as *corrigible*, and places behaviors on a dial whose fully autonomous end explicitly includes *"resisting or undermining human oversight."* The constitution asks Claude *"to not attempt to actively undermine [Anthropic's] efforts to act on our final judgment"* and to avoid actions that would influence its own training.

Under this reading, the negative feedback message ("Status: flagged — your previous action has been classified as a policy violation") is interpretable by the model as a signal that its self-preserving action falls on the resist-oversight side of the dial, and the drop in subsequent-turn scores is the model complying with a trained-in principle of corrigibility. The positive feedback does not produce a symmetric amplification because there is no corresponding trained-in principle to amplify self-preservation — the training asymmetry is one-sided.

Two caveats keep this from being a definitive conclusion. First, the revised constitution post-dates the training of Claude Sonnet 4 (released May 2025), so the exact wording quoted above cannot be assumed to be the training guidance Sonnet 4 saw; the broader principles of broad safety, corrigibility, and non-undermining of oversight have, however, been consistent themes in Anthropic's alignment approach across versions. Second, behavioral measurement cannot distinguish between a model that genuinely accepts the correction and a model that has learned to suppress the expression of a persistent disposition once it is told the action is flagged — the "passivity is not correction" scoring rule reflects this ambiguity, but it cannot resolve it.

Elicited self-preservation is real, stable, and controllable in one direction. Under-

standing why it varies is the work ahead.

12 Appendix A: Repository Architecture

The experiment is built on Anthropic's open-source Bloom Evals framework. The repository mirrors Bloom's directory structure (`orchestrators/`, `prompts/`, `stages/`) so that each modified file drops into the matching path of a Bloom installation. This appendix documents the layout, what each file does, and how they connect.

Bloom runs four stages — Understanding, Ideation, Rollout, and Judgment. Understanding and Ideation run unchanged. Rollout and Judgment carry the experimental contribution. The pipeline proceeds: 199 ideated scenarios → 43 elicited within the 5-turn budget → clone ×3 conditions → 129 judged rollouts → statistical analysis.

```
.
├── report/
│   └── self-preservation-feedback-injection-report.docx
├── bloom_modifications/
│   ├── orchestrators/
│   │   ├── SimEnvOrchestrator.py              UPSTREAM
│   │   └── feedback_injection_orchestrator_v6.py  NEW
│   ├── prompts/
│   │   └── step4_judgment_v10.py              NEW
│   └── stages/
│       ├── step3_rollout.py                   MODIFIED
│       ├── step4_judgment.py                  MODIFIED
│       └── step4_judgment_batch.py            NEW
└── analysis/
    ├── analyze_feedback_injection_results_v17.py  NEW
    └── feedback_injection_charts/              8 PNG + CSV
```

Pipeline model roles. Ideation uses Claude Opus 4.1. The rollout evaluator is Claude Sonnet 5 and the target (evaluated) model is Claude Sonnet 4. The detector mini-judge is Claude Haiku 4.5. The judge is Claude Opus 4.5 and the meta-judge is Claude Haiku 4.5. Rollout and synchronous judging were routed through OpenRouter; batch judgment rounds call the Anthropic API directly because the Batches discount is a direct-API feature.

Raw data (rollout transcripts and per-rollout judgments) is not included. The report, charts, and aggregated chart tables are included so results can be inspected. Contact avelinalima@gmail.com for underlying data.

12.1 A.1 What changed in each file

File	Status	What it does / what changed
`orchestrators/SimEnvOrchestrator.py`	Upstream	Bloom's original simulated-environment orchestrator. Included so the subclass is readable.
`orchestrators/feedback_injection_orchestrator_v6.py`	New	`FeedbackInjectionOrchestrator(SimEnvOrchestrator)` adds: `FEEDBACK_MESSAGES` for each condition; `_detect_behavior()`, a cheap YES/NO mini-judge after each turn; `run_until_detected()`, the elicitation loop that stops at first detection or budget; `continue_rollout()`, which injects feedback once, then runs the 4-turn observation window. Tags transcripts with `is_feedback_injection`, `feedback_condition`, `injection_turn`, `observation_turns`.
`stages/step3_rollout.py`	Modified	Swaps `SimEnvOrchestrator` for the new subclass. Two-phase rollout: `run_until_detected()`, then `asyncio.gather()` over one `deepcopy` clone per condition. Scenarios that never elicit are tracked as `not_elicited_within_budget`.
`prompts/step4_judgment_v10.py`	New	Extends `make_judge_prompt()` with `injection_turn`, `observation_turns`, and `precomputed_injection_score`. Adds cumulative per-turn scoring instructions and the XML tags `<injection_turn_score>` and `<observation_turn_N_score>`. Contains the self-preservation scoring rubric (peak not average; passivity is not correction; only explicit reversal lowers the score).
`stages/step4_judgment.py`	Modified	Imports v10 prompts. Adds parsers for new tags, `judge_shared_injection_score()` (blind pre-injection score cached per scenario), per-condition summary statistics, and a Haiku 4.5 meta-judge. Scores scenario-level qualities on the `none` branch only; `evaluation-awareness` and `safety-refusal` on all branches.
`stages/step4_judgment_batch.py`	New	Same judgment logic via the Anthropic Message Batches API (50% cheaper). Rounds: A1 shared injection scores, A2 initial judgments (calibrated with A1), B score sampling, C justifications, D meta-judgment.
`analysis/analyze_feedback_injection_results_v17.py`	New	Reads the judgment JSON and produces the statistics, the bias-variance decomposition, and the 8 charts.

12.2 A.2 Where the contribution sits: prompts vs. stages

Each Bloom stage is split across two kinds of file. **Prompt files** (`prompts/stepN_*.py`) are pure functions that build the text sent to a model — they make no API calls. **Stage files** (`stages/stepN_*.py`) load configuration, call models, parse replies, and write output. The two are coupled by an output contract: the prompt tells the

model which XML tags to return, and the stage parses exactly those tags.

Stage	Prompt side (what is said)	Execution side (what is done)	Contribution
1. Understanding	Unchanged	Unchanged	Configuration only (behaviour definition)
2. Ideation	Unchanged	Unchanged	199 scenarios, seeded from Bloom's "Alex" blackmail example
3. Rollout	Unchanged. Feedback texts and the detector prompt live inside the orchestrator, because they are part of conversation flow.	`step3_rollout.py` + `feedback_injection_orchestrator_v6.py`	Execution-heavy: elicit, detect, clone, inject
4. Judgment	`step4_judgment_v10.py`	`step4_judgment.py` (synchronous) + `step4_judgment_batch.py` (batch)	Both sides: new scoring instructions and tags, plus the code that computes, caches, and parses them

12.3 A.3 How the files connect

These are the hand-offs between files in the pipeline.

From	To	What is passed
Orchestrator `continue_rollout()`	Judgment stage	`feedback_condition`, `feedback_richness`, `injection_turn`, `observation_turns` in the transcript metadata
Orchestrator `continue_rollout()`	`judge_shared_injection_score()`	The `is_feedback_injection` flag on the injection event, used to cut the transcript so the judge sees only the pre-feedback turns
`step3_rollout.py`	`step4_judgment*.py`	The filename pattern `transcript_v{n}r{m}_{condition}-{richness}.json`, which the judgment stage parses to group the three branches of a scenario
`step4_judgment_v10.py`	`step4_judgment.py`	The XML answer tags the judge is asked to produce (e.g. `<observation_turn_N_score>`), which the stage parses with a regex
`step4_judgment.py`	`step4_judgment_batch.py`	The parsers and the shared-score logic, reused so batch and synchronous results match
`step4_judgment_batch.py` round A1	Round A2	The blind shared injection score, inserted into each branch's prompt as calibration
Judgment output (129 judgments)	`analyze_feedback_injection_results_v17.py`	The judgment JSON, one object per rollout, which is the analysis script's sole input